

archivix

Installation & User Guide

A parallel archive engine for zip, tar.gz, tar.zst, tar.xz, tar.bz2, tar.lz4, 7z and rar. Create, extract, list, read and search archives using every CPU core, from the command line, a desktop app, or an AI agent.

Version 0.1.0

Licensed MIT OR Apache-2.0 · https://github.com/swapnil-saal/archivix

Contents

1. What archivix is
2. Installation: macOS, Windows, Linux, from source
3. Quick start
4. Command reference
5. Patterns, levels and choosing a format
6. The desktop app
7. Using archivix from scripts and AI agents
8. Troubleshooting

1. What archivix is

archivix is one tool for every common archive format. It is built for speed: compression and extraction run on all CPU cores, and archives it creates in `.tar.gz`, `.tar.zst` and `.tar.lz4` carry a small hidden index that makes listing and reading single files instant. Those archives stay fully compatible with the standard tools (`tar`, `gzip`, `zstd`, `lz4`).

It comes in three forms that share the same engine:

- **archivix**: the command-line tool (this guide's main subject).
- **archivix-gui**: a small desktop app for macOS, Windows and Linux.
- **archivix mcp**: a Model Context Protocol server so AI agents can look inside archives.

Supported formats

Format	Create	Extract	Listing
zip (also jar, apk, docx, whl, …)	yes (parallel)	yes, incl. encrypted	instant
tar	yes	yes	instant
tar.zst / tar.gz / tar.lz4	yes (parallel)	yes	instant for archivix-made files
tar.xz / tar.bz2	yes (multithreaded)	yes	reads the stream
tar.br / tar.lzma	yes	yes	reads the stream
7z	yes (multithreaded LZMA2)	yes, incl. encrypted	instant
rar (v1.5–v5)	no (proprietary format)	yes	instant
single files: .gz .zst .xz .bz2 .lz4 .br	yes	yes	—

Formats are recognised by their content, not their file extension, so mislabelled files still open.

2. Installation

Prebuilt downloads are on the releases page: `https://github.com/swapnil-saal/archivix/releases`. Each download has a matching `.sha256` checksum file.

You have	Download
Mac (Apple Silicon or Intel), desktop app	`archivix-0.1.0-macos-universal.dmg`
Mac, command line, Apple Silicon	`archivix-v0.1.0-aarch64-apple-darwin.tar.gz`
Mac, command line, Intel	`archivix-v0.1.0-x86_64-apple-darwin.tar.gz`
Windows 10/11 (64-bit)	`archivix-v0.1.0-x86_64-pc-windows-msvc.zip`
Linux x86_64	`archivix-v0.1.0-x86_64-unknown-linux-gnu.tar.gz`
Linux ARM64 (e.g. Raspberry Pi 4/5, Graviton)	`archivix-v0.1.0-aarch64-unknown-linux-gnu.tar.gz`

The Windows and Linux downloads contain both the command-line tool (`archivix`) and the desktop app (`archivix-gui`).

macOS

Desktop app

Open the `.dmg` and drag **archivix** onto the **Applications** folder.

> **First launch.** The app is signed but not notarized by Apple, so the first launch of a downloaded copy is blocked. Right-click the app in Applications and choose **Open**, then **Open** again. Or run once in Terminal: `xattr -dr com.apple.quarantine /Applications/archivix.app`

Command-line tool

Download the `.tar.gz` for your Mac, then in Terminal:

```
cd ~/Downloads
tar -xzf archivix-v0.1.0-aarch64-apple-darwin.tar.gz
cd archivix-v0.1.0-aarch64-apple-darwin
xattr -d com.apple.quarantine archivix      # allow the downloaded binary to run
sudo install -m 755 archivix /usr/local/bin/
archivix --version
```

Windows

- Right-click the `.zip` and choose **Extract All**.
- Double-click `archivix-gui.exe` to start the desktop app.
- If SmartScreen says *Windows protected your PC*, click **More info**, then **Run anyway** (the app is not code-signed).

To use the command line from any terminal, copy `archivix.exe` to a folder on your PATH. In PowerShell, inside the extracted folder:

```
$dir = "$env:USERPROFILE\bin"
New-Item -ItemType Directory -Force $dir | Out-Null
Copy-Item archivix.exe $dir
$user = [Environment]::GetEnvironmentVariable("Path", "User")
[Environment]::SetEnvironmentVariable("Path", "$user;$dir", "User")
# open a new terminal, then:
archivix --version
```

Linux

Install for your user (no root needed), including a menu entry and icon for the desktop app:

```
tar -xzf archivix-v0.1.0-x86_64-unknown-linux-gnu.tar.gz
cd archivix-v0.1.0-x86_64-unknown-linux-gnu
install -D -m 755 -t ~/.local/bin archivix archivix-gui
install -D -m 644 archivix.desktop ~/.local/share/applications/archivix.desktop
install -D -m 644 archivix.png ~/.local/share/icons/hicolor/256x256/apps/archivix.png
archivix --version        # if not found, add ~/.local/bin to your PATH
```

The binaries need only the standard C/C++ runtime. The desktop app also needs a graphical session (X11 or Wayland).

From source (any platform)

Requires Rust 1.93 or newer (`https://rustup.rs`) and a C/C++ compiler (Xcode Command Line Tools, Visual Studio Build Tools, or gcc/clang).

```
# command-line tool
cargo install --git https://github.com/swapnil-saal/archivix
# desktop app
cargo install --git https://github.com/swapnil-saal/archivix archivix-gui

# or from a clone (builds target/release/archivix and archivix-gui):
git clone https://github.com/swapnil-saal/archivix && cd archivix
cargo build --release --workspace
gui/macos/bundle.sh       # macOS only: build archivix.app and a .dmg
```

> **RAR support.** RAR extraction compiles RARLAB's freeware UnRAR code, which is not under an open-source license. To build without it, add `--no-default-features`. Everything else keeps working.

Verify a download (optional)

```
# macOS / Linux: prints "OK" when the file is intact
shasum -a 256 -c archivix-v0.1.0-x86_64-unknown-linux-gnu.tar.gz.sha256

# Windows (PowerShell): compare the hash with the .sha256 file
Get-FileHash archivix-v0.1.0-x86_64-pc-windows-msvc.zip -Algorithm SHA256
```

3. Quick start

Five commands cover most use. The format of a new archive follows the name you give it.

```
archivix c project.tar.zst src/ docs/    # create (fast, instant listing)
archivix ls project.tar.zst              # list what is inside
archivix cat project.tar.zst src/main.rs # print one file without extracting
archivix x project.tar.zst -C out/       # extract into out/
archivix t project.tar.zst               # verify integrity
```

In a terminal, long operations show a live progress line with percentage, speed, file count and time remaining. Each command ends with a one-line summary, for example:

```
project.tar.zst: 12125 entries, 585.1 MiB -> 238.7 MiB (40.8%) in 1.36s (430.2 MiB/s)
```

Every command has a short alias: `c` create, `x` extract, `ls` list, `t` test, `z` compress, `d` decompress.

4. Command reference

Run `archivix <command> --help` for the same information in your terminal. These options work with every command:

Global option	Meaning
`-j N`, `--threads N`	Use N worker threads (default: all CPU cores).
`-q`, `--quiet`	Hide the summary line.
`-h`, `--help` / `-V`, `--version`	Show help / version.

create (alias: c)

Create an archive from files and folders. The format is chosen from the output name: `.zip`, `.tar`, `.tar.zst`, `.tar.gz`/`.tgz`, `.tar.xz`, `.tar.bz2`, `.tar.lz4`, `.tar.br`, `.7z`.

```
archivix create <OUTPUT> <INPUTS>... [options]
```

Option	Meaning
`-l`, `--level N`	Compression level, in the codec's own scale (see section 5).
`-x`, `--exclude GLOB`	Skip matching files or folders; repeatable.
`-C`, `--cd DIR`	Read inputs relative to DIR, like `tar -C`.
`-F`, `--format FMT`	Force a format instead of using the extension.
`-m`, `--method M`	zip only: `deflate` (default), `zstd`, `store`, `bzip2`, `xz`.
`-L`, `--follow`	Follow symbolic links instead of storing them.
`--chunk SIZE`	Parallel chunk size, e.g. `4M` (advanced).
`--no-index`	Do not add the archivix index to gz/zst/lz4 archives.
`--no-smart-store`	zip: compress everything, even photos/videos that will not shrink.

```
archivix c backup.zip ~/Documents
archivix c site.tgz -C /var/www html        # stores html/... (not /var/www/...)
archivix c src.tar.zst . -x node_modules -x .git -x '*.o'
archivix c small.tar.xz data/ -l 9
archivix c photos.7z Pictures/
```

extract (alias: x)

Extract everything, or only the given names, folders or glob patterns. Paths that would escape the destination folder are refused.

```
archivix extract <ARCHIVE> [MEMBERS]... [options]
```

Option	Meaning
`-C`, `--dest DIR`	Destination folder (default: current folder).
`--strip-components N`	Drop the first N folders from each path.

Option	Meaning
`-k`, `--keep`	Do not overwrite existing files.
`-p`, `--password PW`	Password for encrypted zip or 7z.
`--no-perms`	Do not restore file permissions.
`--json`	Print a JSON summary.

```
archivix x project.zip -C out/
archivix x project.tar.zst 'src/**/*.rs'            # only Rust files under src/
archivix x project.tar.gz docs                      # a whole folder
archivix x release.tgz --strip-components 1
archivix x secret.7z -p 'my password'
```

list (aliases: ls, l)

List the contents without extracting. Optionally filter by names or patterns.

```
archivix list <ARCHIVE> [MEMBERS]... [-l] [--json] [-p PW]
```

Option	Meaning
`-l`, `--long`	Show permissions, size, packed size and date.
`--json`	Machine-readable output.

```
archivix ls project.tar.zst
archivix ls -l project.zip '*.md'
archivix ls --json project.7z > contents.json
```

cat

Write the contents of one or more files to standard output, without extracting the archive. For archivix-made tar.gz/tar.zst only the needed part is decompressed.

```
archivix cat <ARCHIVE> <MEMBERS>... [-p PW]
```

```
archivix cat project.tgz README.md
archivix cat logs.zip app/today.log | grep ERROR
```

grep

Search text files inside an archive for a regular expression. Binary files are skipped. Exits with status 1 when nothing matches.

```
archivix grep <PATTERN> <ARCHIVE> [MEMBERS]... [options]
```

Option	Meaning
`-i`, `--ignore-case`	Case-insensitive match.
`-m`, `--max N`	Stop after N matches (default 1000).
`--json`	Machine-readable results.

```
archivix grep 'TODO|FIXME' project.tar.zst '*.rs'
archivix grep -i 'password' configs.zip
```

info

Show the format, number of files and folders, sizes, compression ratio, and whether instant listing, random access and parallel decoding are available.

```
archivix info <ARCHIVE> [--json] [-p PW]
```

```
archivix info project.tar.gz
```

test (alias: t)

Decode every file and verify its checksum (CRC-32 or the stream checksum). Exits with status 2 if problems are found.

```
archivix test <ARCHIVE> [--json] [-p PW]
```

```
archivix t download.zip && echo OK
```

compress (alias: z)

Compress a single file (not an archive) using all cores.

```
archivix compress <FILE> [-o OUT] [-c CODEC] [-l N] [--rm]
```

Option	Meaning
`-c`, `--codec C`	`zstd` (default), `gzip`, `xz`, `bzip2`, `lz4`, `brotli`, `lzma`.
`-o`, `--output PATH`	Output path (default: input name plus extension).
`--rm`	Delete the input after success.

```
archivix z huge.log                 # -> huge.log.zst
archivix z huge.log -c gzip -l 9    # -> huge.log.gz
```

decompress (alias: d)

Decompress a single compressed file. The codec is detected automatically.

```
archivix decompress <FILE> [-o OUT | -o -] [--rm]
```

Option	Meaning
`-o -`	Write to standard output.
`--rm`	Delete the compressed file after success.

```
archivix d huge.log.zst             # -> huge.log
archivix d data.json.gz -o - | jq .
```

`formats / mcp`

`formats` prints every supported format and what archivix can do with it (`--json` available). `mcp` starts the AI-agent server described in section 7.

```
archivix formats [--json]
archivix mcp
```

```
archivix formats
```

5. Patterns, levels and choosing a format

Selecting files

`extract`, `list` and `grep` accept member names in three forms:

You write	It matches
`docs/guide.md`	exactly that file
`docs`	the folder and everything inside it
`*.rs`	every `.rs` file at any depth (a name-only glob)
`src/**/*.rs`	`.rs` files anywhere under `src/`
`img/photo-?.png`	`?` matches one character; `[abc]` and `{a,b}` also work

Quote patterns (`'*.rs'`) so your shell does not expand them first.

Compression levels

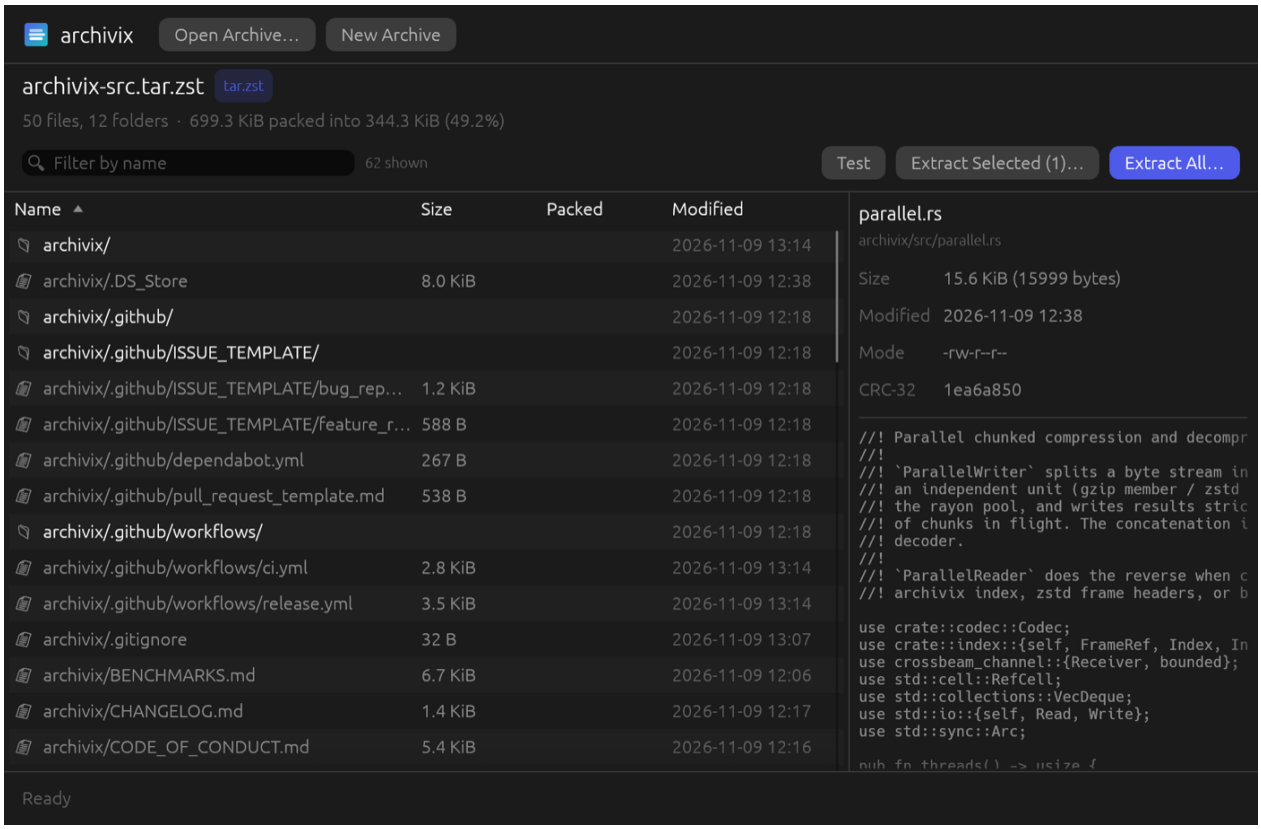
Codec	Range	Default	Notes
zstd (tar.zst, zip -m zstd)	1–22	3	19+ is slow; negative levels (-7..-1) are even faster
deflate (zip, tar.gz)	0–9	6	0 = store only
xz (tar.xz)	0–9	6	highest ratio of the common formats
7z (LZMA2)	0–9	6	
bzip2 (tar.bz2)	1–9	9	
brotli (tar.br)	0–11	6	
lz4 (tar.lz4)	—	—	one fast level

Which format should I use?

If you want…	Use
the best all-round speed, instant listing and single-file reads	`.tar.zst`
something everyone can open (Windows, macOS Finder, phones)	`.zip`
maximum compatibility with Linux tooling	`.tar.gz`

If you want…	Use
the smallest possible file and time is not critical	`.tar.xz` or `.7z`
the fastest possible compression	`.tar.lz4`

6. The desktop app



Browsing an archive, with a file selected and previewed on the right.

Open and browse

- Drag an archive onto the window, or click **Open Archive…**.
- Click a column header to sort; click again to reverse. Type in **Filter by name** to narrow the list.
- Click to select; **Cmd**-click (macOS) or **Ctrl**-click adds or removes items; **Shift**-click selects a range.
- Select a single file to **preview** it on the right: text as-is, anything else as a hex dump.

Extract and test

- **Extract All…** or **Extract Selected (n)…** asks for a folder and extracts into a subfolder named after the archive.
- When it finishes, **Show in Finder / Explorer** opens the result.
- **Test** verifies every file's checksum without writing anything.

Create an archive

- Drag files and folders onto the window (or use **New Archive**, then **Add Files…** / **Add Folder…**).
- Choose a **Format** (hover a choice for advice) and a **Level**.
- **Save as** shows where the archive will go; click **Change…** to pick another location.
- Keep **Skip system files** on to leave out `.DS_Store`, `Thumbs.db` and `__MACOSX`.
- Click **Create Archive**. The progress bar shows speed and file count; **Cancel** stops it.

7. Using archivix from scripts and AI agents

JSON output

`list`, `info`, `test`, `grep` and `extract` accept `--json` and print one JSON document on standard output. The progress bar is never printed when output is not a terminal.

```
$ archivix ls --json project.zip 'README*'
{"count":1,"entries":[{"crc32":3473929125,"kind":"file","mode":420,
  "mtime":1794223761,"path":"README.md","size":4300, ...}],"format":"zip"}
```

MCP server

`archivix mcp` speaks the Model Context Protocol over standard input/output, so assistants such as Claude can inspect archives without extracting them.

Claude Code

```
claude mcp add archivix -- archivix mcp
```

Other MCP clients (JSON configuration)

```
{
  "mcpServers": {
    "archivix": { "command": "/usr/local/bin/archivix", "args": ["mcp"] }
  }
}
```

Tool	What it does
`archive_list`	List members, with a glob filter and pagination.
`archive_info`	Format, counts, sizes, ratio and capabilities.
`archive_read`	Read one member (text, or a hex preview), paged by offset and size.
`archive_grep`	Regex search across text members.
`archive_extract`	Extract all or selected members to a folder.
`archive_create`	Create an archive from paths.

8. Troubleshooting

macOS: "archivix can't be opened" or "cannot be verified"

The download is not notarized. Right-click the app and choose **Open**, or run `xattr -dr com.apple.quarantine /Applications/archivix.app`. For the command-line binary: `xattr -d com.apple.quarantine /usr/local/bin/archivix`.

Windows: SmartScreen blocks the app

Click **More info**, then **Run anyway**. The binaries are not code-signed.

archivix: command not found

The folder containing `archivix` is not on your PATH. See the installation steps for your platform, then open a new terminal.

"cannot infer archive format"

The output name has no recognised extension. Name it e.g. `out.tar.zst`, or add `-F tar.zst`.

"entry is encrypted: pass --password"

Add `-p 'your password'`. A wrong password reports "provided password is incorrect".

I want to create a .rar file

Not possible: RAR's compressor is proprietary. Use `.7z` or `.zip` instead.

Listing a .tar.gz from another tool is slower than one made by archivix

Expected. Without the archivix index the whole stream must be decompressed to list it (archivix still does this faster than `tar`). Re-pack it with archivix to get instant listing.

No progress bar appears

It only appears when output goes to a terminal and the operation takes longer than about 0.2 seconds; `-q` hides it.

Double-clicking an archive in Finder does not open it in the app

Drag the archive onto the window or the Dock icon instead, or use **Open Archive…**.

Linux: the desktop app does not start

It needs a graphical session (X11 or Wayland) and `libxkbcommon`, which desktop distributions include.

Report problems or ask questions at `https://github.com/swapnil-saal/archivix/issues`. Security issues: see SECURITY.md in the repository.